

Introduction to Ubuntu and the Terminal

In this exercise, you will learn how to use the **terminal** in Ubuntu. You do not need any previous experience with Linux or the command line.

Don't worry if you do not understand everything. We will also go over this in the Introduction to Information Science course. Just follow the instructions and experiment!

1. What is the terminal?

Normally, you use a computer by clicking on things. You open folders by clicking on them, create files using menus, and move files by dragging them.

The terminal gives you another way to interact with your computer. Instead of clicking, you type instructions. These instructions are called commands.

For example, instead of opening a folder by clicking on it, you can type a command that tells the computer to go to that folder.

Open the Terminal application in Ubuntu.

You will see a line of text with a cursor at the end. It may look something like this:

```
student@computer:~$
```

The exact text will be different on different computers. The \$ indicates that the terminal is ready for a command. The text before it gives you some information about your user account and your current location.

You do not type the \$ yourself.

When instructions show:

```
$ pwd
```

you only type:

```
pwd
```

and then press Enter.

2. Where am I?

Your computer stores files inside directories. A directory is the same basic idea as a folder.

At any moment, the terminal is working from one particular directory. This is called your current working directory.

To find out which directory you are currently in, type:

```
pwd
```

pwd stands for print working directory.

You should see something similar to:

```
/home/student
```

This is a path. A path describes the location of a file or directory on your computer.

Your personal directory is called your home directory. It is where your own files and folders are normally stored.

The symbol below is a short way of referring to your home directory:

```
~
```

3. What is inside this directory?

To see the files and directories in your current location, type:

```
ls
```

ls is short for list.

The terminal will show the names of the files and directories that are currently there.

Try:

```
ls
```

You may see directories such as:

```
Desktop
Documents
Downloads
Pictures
```

The exact contents depend on the computer you are using, of course.

4. Creating a new directory

We will create a directory for this exercise. Type:

```
mkdir terminal-exercise
```

mkdir stands for make directory.

The first part is the command:

```
mkdir
```

The second part tells the command what directory to create:

```
terminal-exercise
```

Information that you give to a command like this is often called an argument.

So the full command basically means:

Make a directory called terminal-exercise.

Now type:

```
ls
```

You should now see the new directory in the list!

5. Moving into a directory

Creating a directory does not automatically move you into it. To move into the new directory, type:

```
cd terminal-exercise
```

cd stands for change directory.

It changes the directory that the terminal is currently working in. In a graphical file manager, this is similar to double-clicking a folder to open it.

Check your location again:

```
pwd
```

The path should now end with:

```
/terminal-exercise
```

You can also try:

```
ls
```

Nothing may appear. That is normal: you have just created this directory, so it is empty.

6. Going back

There are several useful ways to move between directories.

To move to the directory one level above your current directory, use:

```
cd ..
```

The two dots (..) mean the parent directory, or the directory one level above the current one.

Check where you are:

```
pwd
```

Then go back into your exercise directory:

```
cd terminal-exercise
```

You can return to your home directory from almost anywhere by typing:

```
cd
```

7. Saving time with the Tab key

Typing complete file and directory names can become annoying, especially when names are long.

But good news: the terminal can often complete names for you!

For example, if terminal-exercise exists in your current directory, you can type:

```
cd term
```

and then press Tab.

If there is only one matching name, the terminal completes it:

```
cd terminal-exercise
```

This is called tab completion and is very useful.

8. Creating a text file

Now we will create a text file using a program called Nano. Make sure you are inside your terminal-exercise directory by using the appropriate cd commands you just learned.

Then type:

```
nano myfile.txt
```

Here:

- nano starts the Nano text editor.
- myfile.txt is the file you want Nano to open.

If the file does not exist yet, Nano creates it!

Nano runs inside the terminal. The terminal has not disappeared: it is just currently being used by Nano.

Type three short lines of text, for example:

```
This is my first text file.  
I created it from the terminal.  
Ubuntu is using Linux.
```

To save the file, press:

```
Ctrl + O
```

Nano will ask you to confirm the filename. Press Enter.

To leave Nano, press:

```
Ctrl + X
```

You should now be back at the normal terminal prompt.

9. Checking that the file exists

Type:

```
ls
```

You should now see:

```
myfile.txt
```

10. Looking at the contents of a file

You do not need to open a text editor to read a file! Try:

```
cat myfile.txt
```

cat prints the contents of the file directly in the terminal. You should see the three lines that you wrote.

For a short file, cat is the quickest way to inspect its contents.

11. Counting lines and words

Often, you want to know how large a file is, or if it has content at all.

The command **wc** can count information in a text file. Type:

```
wc myfile.txt
```

wc stands for word count, although it can show more than just the number of words.

Its output includes:

- the number of lines;
- the number of words;
- the number of bytes in the file.

For example, you might see something like:

```
3 15 82 myfile.txt
```

The exact numbers depend on what you wrote. Check whether the number of lines and words looks correct.

You can also ask `wc` for only one type of information. For example:

```
wc -l myfile.txt
```

This shows the number of lines.

12. Commands, arguments and options

At this point, it is useful to look at the general structure of a terminal command. Consider:

```
wc -l myfile.txt
```

This contains three parts:

wc is the command. It tells the computer which program to run.

-l is an option. It changes what the command does.

myfile.txt is an argument. It tells the command which file to work with.

Not every command needs both options and arguments. For example:

```
pwd
```

works by itself, while:

```
cd terminal-exercise
```

uses an argument but no option. You do not need to memorize these terms immediately, but you will see them often. In time, they will become very familiar.

13. Getting help

There are thousands of Linux commands, and most people do not remember all of their options. Luckily, Linux provides manuals for many commands.

Type:

```
man wc
```

man stands for manual.

You will see the manual page for `wc`. You can move through the page using the arrow keys.

Press:

```
q
```

to quit the manual and return to the terminal.

Try another one:

```
man ls
```

There is usually a lot of information there, which you do not need to understand. It is just a useful reference.

14. Reusing previous commands

You do not need to retype previous commands! Just press the Up Arrow key. The terminal shows then your previous command.

Keep pressing the Up Arrow to move further back through your command history. Find:

```
cat myfile.txt
```

and press Enter to run it again. The Down Arrow moves in the other direction through your command history.

This is again a very useful speedup.

15. Searching inside a file

Often, you want to look for a file that contains a certain word. Or you want to check whether a file contains a certain word or not. For this, you can use **grep**.

Choose a word that appears in your file. For example, if your file contains the word terminal, try:

```
grep terminal myfile.txt
```

grep searches the file and prints lines containing the word you searched for.

You can add options to change how it searches. For example:

```
grep -i terminal myfile.txt
```

The -i option makes the search case-insensitive. This means that terminal, Terminal, and TERMINAL are treated as matches.

Another option is -w, which asks grep to match the complete word.

You can combine options:

```
grep -i -w terminal myfile.txt
```

Use the manual to investigate these options yourself:

```
man grep
```

16. Creating a second file

Create another file:

```
nano secondfile.txt
```

Add one short line of text, whatever you want. Save it with:

```
Ctrl + O
```

Press Enter to confirm the filename, then leave Nano with:

```
Ctrl + X
```

Now type:

```
ls
```

You should see both files:

```
myfile.txt  
secondfile.txt
```

17. Removing a file

Files can also be deleted from the terminal. The command is:

```
rm
```

rm stands for remove.

We will remove secondfile.txt. Instead of typing the complete filename, start with:

```
rm sec
```

Then press Tab. The terminal should complete the filename:

```
rm secondfile.txt
```

Press Enter. Now run:

```
ls
```

secondfile.txt should be gone!

Be careful with rm

There is an important difference between deleting a file in a graphical file manager and deleting one with rm.

A file removed with rm **does not** normally go to the Trash. It is simply removed. For that reason, always check your command before pressing Enter.

In particular, be careful whenever you see commands beginning with rm. Do not use rm on files unless you know what you are deleting.

18. Spaces matter

The terminal uses spaces to separate different parts of a command. For example:

```
cat myfile.txt
```

means: run the command cat, and give it the file myfile.txt.

This is different from:

```
catmyfile.txt
```

The computer would try to find a command called catmyfile.txt, which probably does not exist.

Capital letters also matter. For example:

```
myfile.txt
```

```
MyFile.txt
```

These can be two different filenames in Linux. This is called being case-sensitive.

19. What happens if a command is wrong?

Mistakes are normal when using the terminal. Try typing:

```
sl
```

instead of:

```
ls
```

On many systems you will get an error such as:

```
command not found
```

No worries, this does not mean you have damaged anything. It means that the terminal could not find the command you asked it to run.

You may also see messages such as:

```
No such file or directory
```

This usually means that the file or directory name you entered does not exist in the location where you are currently working.

Commands used in this exercise

Command	Meaning	What it does
pwd	print working directory	Shows where you currently are
ls	list	Shows files and directories
mkdir	make directory	Creates a new directory
cd	change directory	Moves to another directory
nano	Nano text editor	Opens or creates a text file
cat	concatenate	Prints the contents of a file
wc	word count	Counts lines, words and bytes
man	manual	Opens the manual for a command
grep	search tool	Searches text
rm	remove	Deletes a file

Symbol or key	Meaning
..	The directory one level above the current directory
Tab	Completes commands or filenames when possible
Up Arrow	Shows previous commands
Ctrl + O	Saves a file in Nano
Ctrl + X	Exits Nano
Ctrl + D	Ends the current terminal session

Final exercise

Without using the graphical file manager, try to do the following:

1. Go to your home directory.
2. Create a directory called terminal-practice.
3. Move into that directory.
4. Check your current location.
5. Create a file called notes.txt using Nano.
6. Write two or three lines in the file and save it.
7. Check that the file exists.
8. Print its contents in the terminal.
9. Count the number of lines in the file.
10. Search for one of its words using grep.
11. Return to your home directory.